Making Single-Threaded Code Multithreaded

Definition

Making single-threaded code multithreaded means redesigning an existing program so that multiple threads can run safely and correctly. This is much harder than simply adding thread creation calls.

1. Why Conversion Is Hard

Main Idea

Many old programs were written for a single thread of control. When multiple threads are introduced, shared data, libraries, signals, and stacks can behave incorrectly unless the program is redesigned carefully.

Key Message

Multithreading is not a small patch on top of single-threaded code. It usually requires major redesign of data handling, libraries, and system-call behavior.

2. Main Problem Areas

Problem Area	Why It Breaks in Multithreaded Code
Global variables	Threads may overwrite each other's shared state
Nonreentrant library routines	Two threads may enter the same routine at once and corrupt internal data
Signals	It becomes unclear which thread should receive or handle them
Stack management	Multiple threads require multiple stacks, which may not be managed automatically

3. Problem 1: Global Variables

Thread-Private Global State

Local variables and parameters usually do not cause problems, but **global variables** do. In particular, some variables are logically global to one thread but should not be shared across all threads.

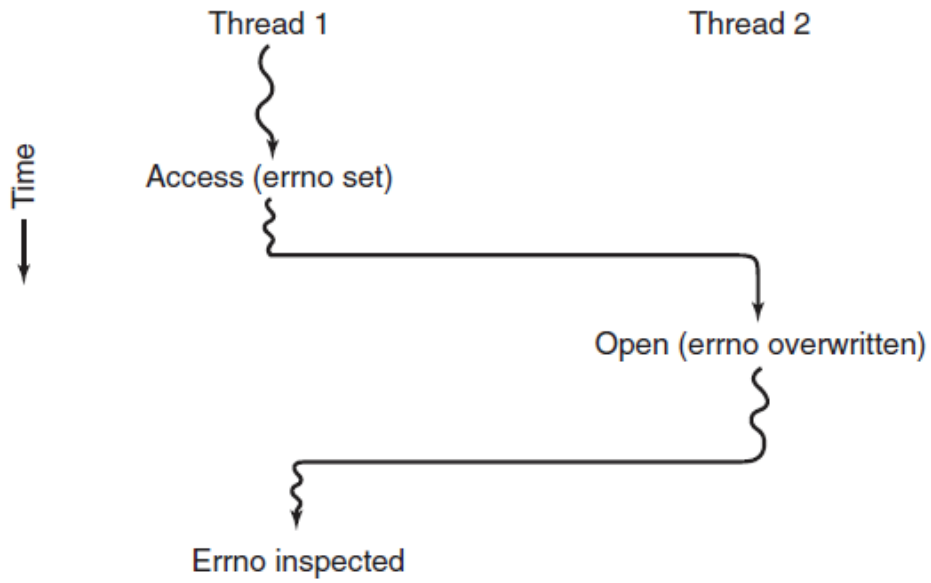

Figure 1: Conflicts between threads over the use of a global variable.

3.1 The `errno` Example

How the Error Happens

In UNIX, when a system call fails, the error code is stored in the global variable `errno`. In a multithreaded program, one thread may set `errno`, then get preempted before reading it. Another thread may then perform a failing system call and overwrite the same variable.

Result

When the first thread resumes, it reads the wrong value of `errno` and behaves incorrectly.

Core Lesson

A variable that worked fine as one process-wide global in single-threaded code may need to become **thread-private** in multithreaded code.

4. Solutions for Global Variables

Possible Solutions

Several approaches can reduce conflicts over globals.

Option 1: Ban Globals

One solution is to forbid global variables entirely. This is theoretically clean, but unrealistic for large existing software systems.

Option 2: Per-Thread Globals

A better approach is to give each thread its own private copy of variables such as `errno`. This creates a new scoping level:

- variables local to one procedure,
- variables visible to all procedures in one thread,
- variables visible everywhere in the program.

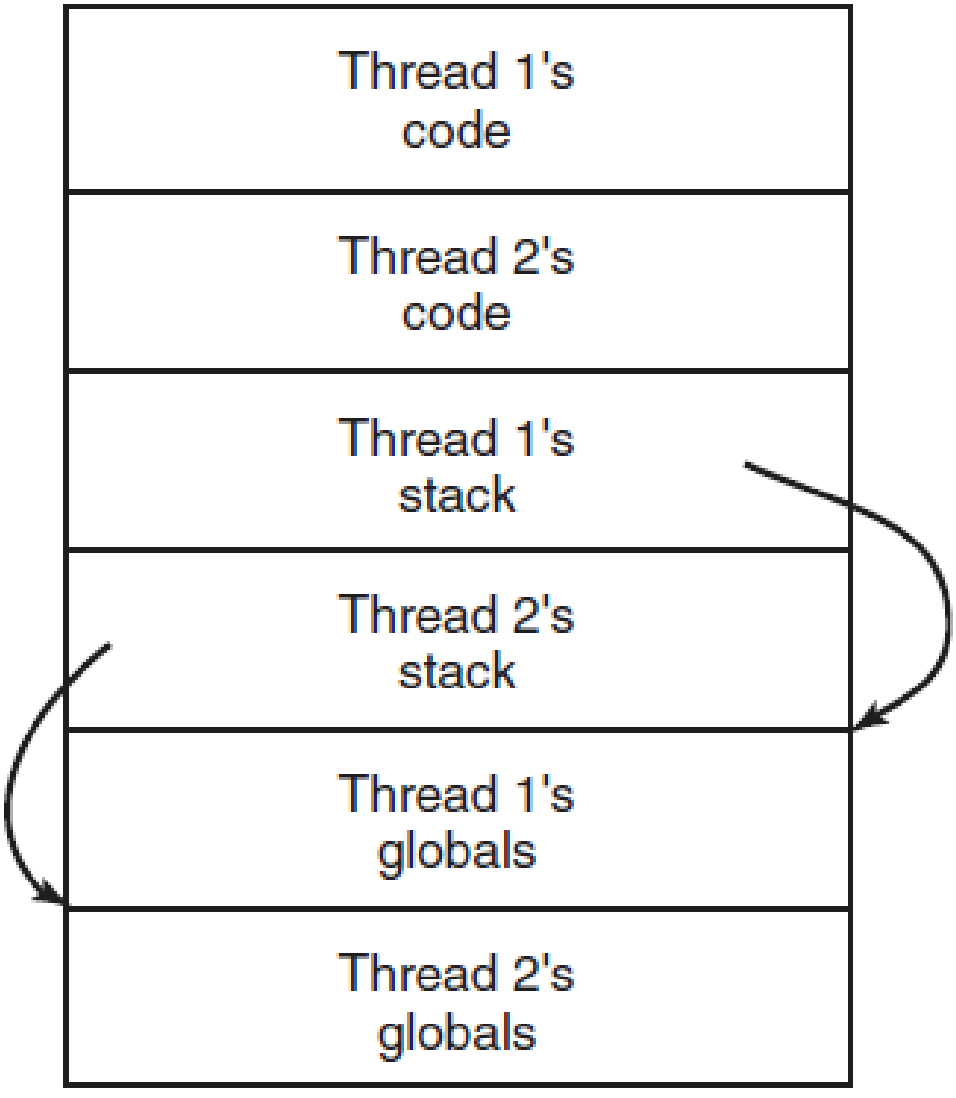

Figure 2: Threads can have private global variables.

4.1 How Per-Thread Globals Can Be Accessed

Method 1: Pass a Globals Structure

One approach is to allocate a block of memory holding the thread's global data and pass it as an extra parameter to all procedures in the thread.

Method 2: Thread-Global Access Routines

Another approach is to introduce special library calls for thread-wide global variables.

Example Interface

```
create_global("bufptr");
set_global("bufptr", &buf);
bufptr = read_global("bufptr");
```

Effect

Each thread that creates a variable like `bufptr` gets its own private storage location, even if another thread uses the same name.

5. Problem 2: Nonreentrant Library Procedures

Nonreentrant Routine

A **nonreentrant** routine is a procedure that was not designed to be entered again before a previous call has finished.

Network Message Example

Suppose a library routine builds an outgoing message in one fixed internal buffer. If one thread fills that buffer and is then interrupted, a second thread may overwrite the same buffer before the first thread finishes.

Memory Allocation Example

Procedures like `malloc` maintain internal data structures such as free-memory lists. If a thread switch happens while those structures are temporarily inconsistent, another thread may use invalid pointers and crash the program.

Why This Is Serious

Fixing these issues may require rewriting major parts of the library to make them reentrant.

6. Possible Fixes for Nonreentrant Libraries

Best Fix: Rewrite the Library

The cleanest solution is to redesign library routines so that they are reentrant and safe for concurrent use.

Wrapper or Jacket Approach

Another solution is to put a **jacket** around each library routine. The jacket marks the library as busy and blocks other threads from entering library code until the current call finishes.

Tradeoff

Jackets can make the code safe, but they greatly reduce parallelism because only one thread can use the protected library at a time.

7. Problem 3: Signals

Why Signals Are Difficult

Signals are already complicated in single-threaded programs. In multithreaded programs, they become even harder because not all signals naturally belong to the whole process in the same way.

Thread-Specific Signals

Some signals logically belong to one thread. For example, if a thread calls `alarm`, it makes sense for the resulting signal to return to that same thread.

User-Level Thread Problem

If threads are implemented entirely in user space, the kernel does not know about them and therefore cannot easily deliver a signal to the correct thread.

Process-Wide Signals

Other signals, such as keyboard interrupt, are not obviously thread specific. This creates difficult questions:

- Should one designated thread handle them?
- Should all threads receive them?
- Should a new pop-up thread handle them?

Conflicting Goals

One thread may want a signal such as `CTRL-C` to terminate the whole process, while another thread may want to catch it and continue running. These wishes are incompatible.

8. Problem 4: Stack Management

Single-Thread Case

In many systems, if a process stack overflows, the kernel automatically provides more stack space.

Multithreaded Case

When a process has multiple threads, each thread needs its own stack.

Why This Becomes Difficult

If the kernel does not know about every thread stack, it may not recognize that a memory fault is actually a request to grow one thread's stack. As a result, automatic stack growth may fail.

9. Big Picture

Main Conclusion of the Section

These problems are not impossible to solve, but they show that converting single-threaded code into multithreaded code requires more than simply adding thread creation. It often requires:

- redefining system-call behavior,
- rewriting libraries,
- redesigning data handling,
- and preserving backward compatibility for programs that still use only one thread.

10. Quick Review

Exam-Focused Points

- Converting single-threaded code to multithreaded code is much harder than it first appears.
- Process-wide globals such as `errno` can cause thread conflicts.
- One solution is to give each thread its own private global variables.
- Many library routines are not reentrant and can break when used by multiple threads at once.
- Jackets or wrappers can protect nonreentrant libraries, but reduce parallelism.
- Signals become harder because it is unclear which thread should receive or handle them.
- Multiple threads require multiple stacks, which complicates stack growth and fault handling.
- A real multithreaded conversion usually requires substantial redesign.